Fixation Disparity Curve Modeling

Technical Documentation of the TFG Repository

Project:	TFG – Phase 1 Web Application
Backend:	Python • FastAPI • GEKKO
Frontend:	React 19 • TypeScript • Vite
Testing:	pytest (backend) • Vitest (frontend)

April 2026

Contents

1 Project Overview

1.1 What the project is

The repository implements **Phase 1** of a TFG (*Trabajo de Fin de Grado*) focused on **Fixation Disparity Curves (FDC)** used in optometry. It is an interactive web application that:

- accepts **7 clinical y-values** measured at fixed x-positions $\{-15, -10, -5, 0, 5, 10, 15\}$;
- fits **four candidate nonlinear models** (T1, T2, T3, T4) to those points;
- computes per-model error metrics (**SSE** and **RMSE**) and the paper-defined **slope** $|f(3) - f(-3)|/6$;
- returns a classification indicating the best-fitting model by SSE and by RMSE;
- visualises the four fitted curves along with the measured points, exposes a classification card (best fit, slope, fixation disparity, associated phoria), offers a chart hover readout, an export menu with high-resolution PNG and PDF report output, and displays a collapsible metrics table.

The reference paper is bundled under `docs/references/`; its public URL is cited in `docs/README.md`: https://pmc.ncbi.nlm.nih.gov/articles/PMC12682111/pdf/OPO-45-1642.pdf.

1.2 Main purpose

Phase 1 delivers an end-to-end, stateless tool for clinicians and researchers to approximate a patient's FDC, quantify model fit, and pick the curve type that best describes the measurement.

1.3 Tech stack

Layer	Technology (as found in repo files)
Backend runtime	Python 3.x (*no explicit pin*; tested with 3.13)
Web framework	FastAPI `0.129.2`
ASGI server	Uvicorn `0.41.0`
Validation	Pydantic `2.12.5`
Numerics	NumPy `2.4.2`
Optimisation solver	GEKKO `1.3.2`
Backend tests	pytest `9.0.3` + httpx `0.28.1` (`backend/requirements-dev.txt`)
Frontend framework	React `19.2.0` + React-DOM `19.2.0`
Language	TypeScript `~5.9.3` (strict mode)
Build tool	Vite `7.3.1` (`@vitejs/plugin-react 5.1.1`)
Charting	Recharts `3.7.0`
PDF generation	jsPDF `4.2.1`
Linter	ESLint `9.39.1` + `typescript-eslint 8.48.0`
Frontend tests	Vitest `4.1.4` + jsdom + Testing Library (React, jest-dom, user-event)
Persistence	*None* (stateless API)
Container / CI	*None present in repository*

1.4 Features

- Seven-point input panel with clinical presets (40 cm, 25 cm) and a custom-distance option.
- Nonlinear fitting of four models with per-model parameter constraints, solved by GEKKO.

- Per-model metrics: SSE, RMSE, paper-defined slope.
- Classification by both SSE and RMSE (`best_by_sse`, `best_by_rmse`).
- Composed chart (four curves + scatter of measured points) with reference lines at $x = 0$ and $y = 0$ and fixed axis domain $[-20, 20]$.
- Hover readout panel showing interpolated model values and snap-to-measured behaviour (`lib/chartHover.ts`).
- Patient-limit dashed markers at $x = -15, -10$ when measured $y = 0$ (`lib/chartAnnotations.ts`).
- Classification card showing best fit, slope, fixation disparity (y at $x = 0$) and associated phoria.
- Collapsible metrics table with the best-by-SSE row highlighted.
- Export menu with PNG (2×) and PDF report output (`lib/exportChart.ts`, `lib/pdfReport.ts`, `components/PdfExportDialog.tsx`).
- Version-counter protection in the SPA against stale responses.
- Automated test suite on both sides (26 backend + 56 frontend tests).

2 Repository Structure

2.1 Top-level tree

```
TFG/
|-- backend/
|   |-- app/
|   |   |-- main.py                  # FastAPI entry point
|   |   `-- services/
|   |       `-- fdc_fit.py           # GEKKO curve fitting + metrics
|   |-- tests/
|   |   |-- conftest.py              # Puts backend/ on sys.path
|   |   |-- test_fdc_fit.py          # Unit + integration tests
|   |   `-- test_main.py             # HTTP tests via TestClient
|   |-- pytest.ini                   # pytest config (testpaths, markers)
|   |-- requirements.txt             # Pinned runtime Python dependencies
|   `-- requirements-dev.txt         # Test-time dependencies
|-- frontend/
|   |-- images/UPC_Logo.png          # Header logo asset
|   |-- public/vite.svg
|   |-- src/
|   |   |-- api/fdc.ts               # Backend client: computeFits()
|   |   |-- components/              # React components
|   |   |   |-- AdvancedMetricsSection.tsx
|   |   |   |-- ClassificationCard.tsx
|   |   |   |-- ClinicalReportChart.tsx
|   |   |   |-- CurveChart.tsx
|   |   |   |-- ExportMenu.tsx
|   |   |   |-- HoverReadoutPanel.tsx
|   |   |   |-- InputPanel.tsx
|   |   |   |-- MetricsTable.tsx
|   |   |   |-- PageHeader.tsx
|   |   |   `-- PdfExportDialog.tsx
|   |   |-- constants/fdc.ts         # Fixed x, presets, labels, colours
|   |   |-- hooks/useClickOutside.ts # Dismiss-on-outside-click hook
|   |   |-- lib/
|   |   |   |-- chart.ts             # mergeModelCurves
|   |   |   |-- chartAnnotations.ts  # Patient-limit markers
|   |   |   |-- chartHover.ts        # Hover/interp/snap helpers
```

```
|   |   |   |-- clinicalSummary.ts   # FD at x=0, associated phoria
|   |   |   |-- exportChart.ts       # SVG -> PNG export
|   |   |   |-- input.ts             # Validation helpers
|   |   |   `-- pdfReport.ts         # jsPDF composition
|   |   |-- types/fdc.ts             # Shared TypeScript types
|   |   |-- test/setup.ts            # Vitest setup (jest-dom, cleanup)
|   |   |-- **/*.test.ts{,x}         # Co-located Vitest tests
|   |   |-- App.tsx                  # Root component
|   |   |-- main.tsx                 # React entry point
|   |   |-- App.css
|   |   `-- index.css
|   |-- index.html
|   |-- package.json
|   |-- package-lock.json
|   |-- tsconfig.json
|   |-- tsconfig.app.json
|   |-- tsconfig.node.json
|   |-- vite.config.ts
|   |-- vitest.config.ts             # jsdom env, coverage, setup file
|   |-- eslint.config.js
|   `-- README.md                    # Project README
`-- docs/
    |-- README.md                    # Phase 1 requirements
    |-- Requirements.pdf
    |-- TFG_Documentation.tex        # This document
    |-- model-and-errors.md          # empty file
    `-- references/                  # Reference paper PDF
```

2.2 Purpose of each important folder/file

Path	Purpose
`backend/app/main.py`	FastAPI application: CORS configuration, Pydantic request model, two routes (`/api/v1/health`, `/api/v1/compute`).
`backend/app/services/fdc_fit.py`	Core numerical module: four GEKKO models with constraints, SSE/RMSE helpers, paper-defined slope, JSON assembly.
`backend/tests/test_fdc_fit.py`	Unit + integration tests for the fitter (helpers, evaluation, slope, validation, end-to-end fits against synthetic data).
`backend/tests/test_main.py`	HTTP-level tests using FastAPI's `TestClient`.
`backend/pytest.ini`	pytest configuration: test discovery root and the `slow` marker for GEKKO-backed tests.
`backend/requirements.txt`	Pinned runtime Python dependencies.
`backend/requirements-dev.txt`	Test-time dependencies only (pytest, httpx).
`frontend/src/App.tsx`	Orchestration of the SPA: state, compute trigger, race-safe versioning, layout.
`frontend/src/api/fdc.ts`	Thin fetch wrapper around `POST /api/v1/compute`.
`frontend/src/components/*.tsx`	Presentational components (header, input, chart, classification, metrics, hover readout, export menu, PDF dialog, clinical report chart).
`frontend/src/constants/fdc.ts`	Fixed x-values, axis domain/ticks, viewing-distance presets, model labels and colours.
`frontend/src/hooks/useClickOutside.ts`	Reusable dismiss-on-outside-click and Escape-key hook.

`frontend/src/lib/*.ts`	Pure helper functions (chart merging, annotations, hover interpolation, clinical summary, PNG export, PDF report, input validation).
`frontend/src/types/fdc.ts`	Shared TypeScript types for the API contract and UI.
`frontend/src/test/setup.ts`	Vitest setup file: jest-dom matchers + React Testing Library cleanup.
`frontend/vite.config.ts`	Vite configuration with the `/api` proxy to `http://localhost:8000`.
`frontend/vitest.config.ts`	Vitest configuration (`jsdom`, setup file, coverage).
`frontend/tsconfig*.json`	Strict TypeScript project references for app code and the Vite config.
`frontend/eslint.config.js`	Flat ESLint configuration.
`docs/README.md`	Phase 1 functional requirements.
`docs/references/`	Reference paper used for the slope/error methodology.

3 Prerequisites

3.1 Languages and runtimes

- **Python**: no version is pinned in `requirements.txt` and there is no `pyproject.toml`. The pinned dependency versions (e.g. `numpy==2.4.2`, `pydantic==2.12.5`) imply a recent interpreter; **Python 3.11+** is a safe baseline, and the test suite in this repository has been run successfully under Python 3.13.
- **Node.js**: `frontend/package.json` has no `engines` field. Vite 7 documentation lists Node 20.19+ or 22.12+; the tests in this repository have been run successfully under Node 22.
- **npm**: implied by `package-lock.json`. No pnpm or yarn lock file is present.

3.2 Declared backend runtime dependencies

See `backend/requirements.txt`. Key pinned packages include `fastapi`, `uvicorn`, `pydantic`, `numpy`, and `gekko` (versions listed in Table of Tech Stack above).

3.3 Declared backend test dependencies

From `backend/requirements-dev.txt`:

```
pytest==9.0.3
httpx==0.28.1
```

3.4 Declared frontend dependencies

The `package.json` declares:

- **Runtime**: `react`, `react-dom`, `recharts`, `jspdf`.
- **Dev tooling**: `vite`, `@vitejs/plugin-react`, `typescript`, ESLint and its plugins.
- **Testing**: `vitest`, `@vitest/coverage-v8`, `jsdom`, `@testing-library/react`, `@testing-library/jest-dom` `@testing-library/user-event`.

3.5 Databases / external services

None. The backend is stateless and makes no outbound network calls.

4 Installation and Setup

4.1 Backend

```
cd backend
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
# optional, only if you want to run the tests:
pip install -r requirements-dev.txt
```

4.2 Frontend

```
cd frontend
npm install
```

4.3 Environment variables

The repository defines no environment variables. A search across the codebase for `os.environ`, `process.env` and `import.meta.env` yields no matches; there is no `.env.example`. Instead, the relevant origins are hard-coded:

- Backend CORS allow-list: `http://localhost:5173` (`backend/app/main.py`).
- Frontend dev proxy target: `http://localhost:8000` (`frontend/vite.config.ts`).

4.4 Database

Not applicable — no database layer exists in the repository.

5 How to Run the Project

5.1 Development mode

Run backend and frontend in two terminals.

Terminal 1 — backend:

```
cd backend
# activate venv first
uvicorn app.main:app --reload --port 8000
```

Interactive FastAPI docs are served at http://localhost:8000/docs and http://localhost:8000/redoc by default.

Terminal 2 — frontend:

```
cd frontend
npm run dev
```

The dev server listens on Vite's default port (5173). Requests to `/api/*` are proxied to `http://localhost:8000` (see `vite.config.ts`).

5.2 Production build (frontend)

```
cd frontend
npm run build       # tsc -b && vite build -> frontend/dist/
npm run preview     # local preview of the built artifacts
```

5.3 Production mode (backend)

A common self-hosted pattern is:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

behind a reverse proxy that forwards `/api/*` to the backend and serves `frontend/dist/` as static assets. The deployment section (§13) details a full procedure.

5.4 Docker / Compose

Not found in repository.

6 Scripts and Commands

6.1 Frontend scripts (from `package.json`)

Script	Command	Effect
`dev`	`vite`	Start Vite dev server with HMR (default port 5173).
`build`	`tsc -b && vite build`	Project-reference type-check, then production build into `frontend/dist/`.
`lint`	`eslint .`	Run ESLint over the frontend sources.
`preview`	`vite preview`	Serve the production build locally.
`test`	`vitest run`	Run the full Vitest suite once and exit.
`test:watch`	`vitest`	Watch-mode test runner for development.
`test:coverage`	`vitest run -coverage`	Run tests and emit coverage under `coverage/`.

6.2 Backend scripts

No `Makefile` or `pyproject.toml` scripts exist. The application is started directly with `uvicorn app.main:app` and the test suite with `pytest` from the `backend/` directory.

7 Architecture

7.1 High-level view

The application is a two-tier SPA + REST API:

- **Frontend (Vite + React 19)** runs client-side, owns the input panel, chart, classification and metrics UI, and calls the backend via `fetch`. It uses component-local state (`useState`, `useMemo`, `useRef`) — *no global store.*
- **Backend (FastAPI + Uvicorn)** exposes a tiny, stateless API. Each request rebuilds four GEKKO models, solves them, and returns the fit results as JSON.

7.2 Request flow

1. User selects a viewing distance (`40cm`, `25cm`, or `other`); for presets, the seven input fields are pre-filled from `VIEWING_DISTANCE_PRESETS` in `frontend/src/constants/fdc.ts`.
2. On "Run Statistical Fit", `App.tsx` validates inputs (`validateViewingDistance`, `parseYValues` from `src/lib/input.ts`) and calls `computeFits(values)` (`src/api/fdc.ts`).
3. `computeFits` issues `POST /api/v1/compute` with `{"y": [...]}`.
4. FastAPI's Pydantic layer validates the 7-length vector. The handler calls `fit_all_models(y)` in `backend/app/services/fdc_fit.py`.
5. `fit_all_models` fits T1–T4 through `_fit_single_model`, computes SSE/RMSE/slope, builds smooth curves and the classification, and returns JSON.
6. The frontend stores the response, `mergeModelCurves` assembles it for Recharts, and the components render the chart, classification card, hover readout and advanced metrics table.

7.3 Classification logic (backend)

For each model $t \in \{T1, T2, T3, T4\}$ a GEKKO NLP is solved that minimises the SSE of the model predictions against the measured y-values, subject to the per-model parameter constraints in `_fit_single_model`. The classification is the argmin over SSE (and independently over RMSE).

7.4 Model equations

Model	Equation	Parameters
T1	$f(x) = a + b\,(x-c)^3$	a, b, c
T2	$f(x) = a + b\,\exp(-c\,(x-d))$	a, b, c, d
T3	$f(x) = a - b\,\exp(c\,(x-d))$	a, b, c, d
T4	$f(x) = a - b\,\arctan(c\,(x-d))$	a, b, c, d

7.5 Slope and clinical summary

- **Slope (paper-defined):** $slope = |f(3) - f(-3)|6$, computed analytically per model in `_compute_slope_paper`.
- **Fixation disparity:** value of the best-fit curve at $x = 0$, computed on the client in `frontend/src/lib/clinicalSummary.ts`.
- **Associated phoria:** smallest non-zero x at which the measured data crosses $y = 0$ (same helper).

8 Detailed Codebase Explanation

8.1 Backend: `backend/app/main.py`

Entry point (abridged):

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, conlist

from app.services.fdc_fit import fit_all_models
```

```python
app = FastAPI(title="TFG Fixation Disparity API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class ComputeRequest(BaseModel):
    y: conlist(float, min_length=7, max_length=7)

@app.get("/api/v1/health")
def health():
    return {"status": "ok"}

@app.post("/api/v1/compute")
def compute(req: ComputeRequest):
    try:
        return fit_all_models(req.y)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Computation failed: {e}")
```

Key points:

- CORS is restricted to the Vite dev origin.
- Request validation uses a constrained list of seven floats.
- `ValueError` from the fitter becomes a 400; any other exception becomes a 500 with a message.

8.2 Backend: `backend/app/services/fdc_fit.py`

The module defines (non-exhaustive):

- `DEFAULT_X = [-15, -10, -5, 0, 5, 10, 15]`.
- `@dataclass FitResult` with fields `model`, `params`, `sse`, `rmse`, `slope`, `curve_points`, `fitted_at_x`.
- `_sse`, `_rmse` — numpy-based metric helpers.
- `_fit_error` — GEKKO `Intermediate` SSE objective.
- `_build_smooth_x(x, n=200)` — dense x vector for plotting.
- `fit_all_models(y, x, smooth_n)` — validates input shape and finiteness, runs all four fits, computes `best_by_sse` and `best_by_rmse`, returns the response dictionary.
- `_fit_single_model` — GEKKO model per curve type with per-model constraints.
- `_eval_model` — evaluates the fitted analytical model at arbitrary x values.
- `_compute_slope_paper` — computes $|f(3) - f(-3)|/6$ analytically per model.

8.3 Frontend: `frontend/src/App.tsx`

The root component keeps all state: selected viewing distance, custom distance and its *touched* flag, the 7 y-value strings, the loading/error/response triplet, and a `computationVersionRef` used to discard stale responses when inputs change mid-flight.

8.4 Frontend: components

Component	Responsibility
`PageHeader`	Branded header; title and `images/UPC_Logo.png`.
`InputPanel`	Viewing-distance dropdown; custom-distance input when `other`; seven y-value inputs; submit button; error display.
`CurveChart`	Recharts `ComposedChart` with four `Line` series and a measured-point `Scatter`; reference lines, fixed axis domain $[-20, 20]$.
`HoverReadoutPanel`	Side panel showing the interpolated values for each model at the cursor position.
`ClassificationCard`	Best fit by SSE; slope from the selected model; fixation disparity; associated phoria.
`AdvancedMetricsSection`	Collapsible wrapper around the metrics table.
`MetricsTable`	Model / SSE / RMSE / Slope table; highlights the best-by-SSE row.
`ExportMenu`	Dropdown with PNG and PDF export actions.
`PdfExportDialog`	Modal that collects metadata before the PDF report is generated.
`ClinicalReportChart`	Print-oriented chart used by the PDF export.

8.5 Frontend: library helpers (`src/lib/`)

File	Function(s)
`input.ts`	`validateViewingDistance`, `parseYValues`.
`chart.ts`	`mergeModelCurves`.
`chartAnnotations.ts`	`getPatientLimitMarkers`.
`chartHover.ts`	Hover snapshot / snap / interpolation helpers used by `CurveChart`.
`clinicalSummary.ts`	`deriveClinicalMeasurements` (FD at $x = 0$, associated phoria).
`exportChart.ts`	`renderSvgToPngDataUrl`, `exportSvgToPng`.
`pdfReport.ts`	jsPDF composition for the exported report.

9 API Documentation

9.1 `GET /api/v1/health`

Purpose: liveness probe.

Response 200:

```
{ "status": "ok" }
```

9.2 `POST /api/v1/compute`

Purpose: fit all four FDC models to 7 measured y-values at the fixed x positions $\{-15, -10, -5, 0, 5, 10, 15\}$ and return per-model metrics plus a classification.

Request body:

```
{ "y": [y1, y2, y3, y4, y5, y6, y7] }
```

Constraints: `y` must have exactly seven finite floats.

Response 200 (shape):

```
{
  "x": [-15, -10, -5, 0, 5, 10, 15],
  "measured": [{ "x": -15, "y": 0.0 }, ... ],
  "models": {
    "T1": {
      "params": { "a": 0.0, "b": 0.0, "c": 0.0 },
      "sse":  0.0,
      "rmse": 0.0,
      "slope": 0.0,
      "fitted_at_x": [ 7 floats ],
      "curve": [ { "x": -15.0, "y": 0.0 }, ... 200 points ... ]
    },
    "T2": { "params": { "a": 0, "b": 0, "c": 0, "d": 0 }, ... },
    "T3": { "params": { "a": 0, "b": 0, "c": 0, "d": 0 }, ... },
    "T4": { "params": { "a": 0, "b": 0, "c": 0, "d": 0 }, ... }
  },
  "classification": {
    "best_by_sse":  "T1|T2|T3|T4",
    "best_by_rmse": "T1|T2|T3|T4"
  }
}
```

Error responses:

- `400` — `ValueError` from the fitter (wrong count, non-finite values) bubbles up as `{"detail": "..."}`.
- `422` — Pydantic validation failure of the request body.
- `500` — other exceptions wrapped as `{"detail": "Computation failed: ..."}`.

Interactive API docs are available when the backend is running at http://localhost:8000/docs (Swagger UI) and http://localhost:8000/redoc (ReDoc).

10 Database and Persistence

The application has **no persistence layer**. No ORM, no migrations, no schema files, no seeded data, no per-user state. Each `POST /api/v1/compute` is independent and produces its result in-memory.

11 Testing Strategy

11.1 Overall philosophy

The repository ships two independent test suites that can each be run in isolation. Both suites target *pure logic* first — the numerical fitter, the parsing / merging / clinical-summary helpers, the API contract — and only lightly touch rendering concerns. This keeps the suites fast, deterministic, and valuable as regression guards during deployment.

11.2 Backend — pytest

Test files live in `backend/tests/`. Configuration is in `backend/pytest.ini`, which points `testpaths` at the `tests` folder and registers a `slow` marker for the GEKKO-backed integration fits (the solver run dominates the wall-clock time).

Test-time dependencies are isolated in `backend/requirements-dev.txt` so that a production install of `requirements.txt` stays lean.

Running:

```
cd backend
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pytest                          # all tests
pytest -m "not slow"            # skip solver-backed integration tests
```

What the suite covers:

- `tests/test_fdc_fit.py`:
  - Metric helpers (`_sse`, `_rmse`) on synthetic inputs.
  - Smooth-x grid (`_build_smooth_x`) length and monotonicity.
  - Model evaluation (`_eval_model`) for T1–T4 at representative x values.
  - Paper-defined slope (`_compute_slope_paper`) for closed-form cases (e.g. cubic ⇒ slope = 9).
  - Input validation: wrong length, NaN, Infinity, wrong x-shape.
  - End-to-end `fit_all_models` runs against y-data drawn from T1 with low amplitude; asserts envelope shape, classification well-formedness, and near-zero residual SSE for the matching model.
  - `smooth_n` parameter controls the curve resolution.
  - Measured points pair correctly with `DEFAULT_X`.
- `tests/test_main.py`:
  - `GET /api/v1/health` returns `{"status": "ok"}`.
  - `POST /api/v1/compute` rejects missing body, wrong-length arrays, non-numeric values, and non-finite values (status codes 400 or 422 depending on the validation layer that catches them).
  - Full success call returns the complete JSON envelope with all four models and a well-formed classification block.

At the time of writing this documentation, the backend suite contains **26 tests** and runs in about 2 seconds on a modern laptop.

11.3 Frontend — Vitest

Configured in `frontend/vitest.config.ts` with:

- `environment: "jsdom"` (required for React Testing Library);
- a shared setup file at `src/test/setup.ts` that registers `@testing-library/jest-dom` matchers and runs `cleanup()` between tests;

- a coverage provider (`v8`) emitting `text`, `html` and `lcov` reports under `coverage/`.

Test files are co-located with their source under `src/` using `*.test.ts` and `*.test.tsx` suffixes.

Running:

```
cd frontend
npm test                 # run all tests once
npm run test:watch       # watch mode for development
npm run test:coverage    # generate coverage report
```

What the suite covers:

- `src/lib/input.test.ts`: both branches of `validateViewingDistance` (all preset options + the custom "other" path) and `parseYValues` on numeric, alphabetic, NaN, and Infinity inputs.
- `src/lib/chart.test.ts`: `mergeModelCurves` for null input, full-length curves, and unequal-length safety truncation.
- `src/lib/clinicalSummary.test.ts`: `deriveClinicalMeasurements` including empty input, FD at $x = 0$, the nearest-non-zero phoria rule, and $-0 \to +0$ normalisation.
- `src/lib/chartAnnotations.test.ts`: `getPatientLimitMarkers` across the four zero-pattern cases at $x \in \{-15, -10\}$.
- `src/lib/chartHover.test.ts`: pixel-ratio-to-axis conversion, measured-value lookup, snap-to-measured, exact-match snapshot, linear interpolation between curve points, and snapshot equality.
- `src/constants/fdc.test.ts`: axis domain, fixed x values, model keys, and both preset-distance lookups via `getPresetValuesForFixedX`.
- `src/api/fdc.test.ts`: `computeFits` request shape, JSON `detail` surfacing, raw-text fallback, and the empty-body HTTP-status fallback (all using a mocked `fetch`).
- `src/components/MetricsTable.test.tsx`: placeholder state, per-model rows with 3-decimal metric formatting, and the `metric-table__row-active` highlight for the best-by-SSE row.
- `src/components/ClassificationCard.test.tsx`: placeholder state, selected-model label + slope, fixation disparity at $x = 0$, and the nearest-non-zero associated phoria readout.

At the time of writing this documentation, the frontend suite contains **56 tests** across 9 test files and runs in about 2 seconds with a cold cache.

11.4 What is intentionally *not* covered

- No visual regression tests (no Storybook / Chromatic / Percy integration in the repo).
- No full end-to-end browser tests (no Playwright / Cypress).
- No explicit CI runner; tests are launched manually or by a local commit hook.
- PDF report generation (`pdfReport.ts`) is not unit-tested because jsPDF's output depends on installed fonts and a real canvas; manual smoke-testing is recommended on the target environment.

12 Build, Lint and Type-Checking Workflow

12.1 Type-checking

Run via `npm run build` in `frontend/`, which chains `tsc -b` (project references across `tsconfig.app.json` and `tsconfig.node.json`, both with `strict: true`, `noUnusedLocals`, `noUnusedParameters`, `noFallthroughCasesInSwitch` and `noUncheckedSideEffectImports`) before `vite build`.

12.2 Linting

```
cd frontend
npm run lint
```

Configuration lives in `frontend/eslint.config.js`.

12.3 Formatter

No Prettier or equivalent configuration was found.

12.4 Coverage

Generated on demand via `npm run test:coverage`; output is written to `frontend/coverage/` (not committed to the repository).

13 Deployment Guide

This section documents how to deploy the application. The repository does *not* include deployment automation, containerisation, or pipeline definitions, so deployment is described here as a *manual, self-hosted process* based on a static frontend build and a separately executed FastAPI backend. The procedure is written to be reproducible by anyone with basic Linux server administration skills.

13.1 Supported deployment models

Three deployment models are practical for this project. They are listed in order of increasing operational maturity.

1. **Single-host manual deployment** (Nginx + Uvicorn + systemd on one Linux server). Recommended for academic review and demonstrations. This is the model documented in detail below.
2. **Single-host Dockerised deployment** (a custom Dockerfile for each tier, or a single multi-stage image). Not included in the repo but straightforward to add; deferred to a future phase.
3. **Cloud PaaS deployment** (e.g. frontend on Netlify/Vercel + backend on Fly.io / Railway / Render). Viable because the API is stateless, but requires configuring CORS for the specific hosted origin.

13.2 CI / CD

Not found in repository. There is no `.github/workflows/`, no `.gitlab-ci.yml`, and no other pipeline configuration.

Consequently, every release must currently be validated and deployed manually. The presence of a local test suite (see §11) gives the operator a fast pre-flight check before publishing, and the commands below include an explicit test step.

13.3 Containers

Not found in repository. There is no `Dockerfile`, no `docker-compose.yml`, and no other container orchestration manifest. The deployer must prepare the server environment manually, install both Python and Node.js dependencies, build the frontend, and configure a web server / reverse proxy to connect the frontend and backend.

13.4 Production topology assumed in this document

- **Frontend**: Vite production build served as static files by Nginx.
- **Backend**: FastAPI served by Uvicorn, bound locally on `127.0.0.1:8000`, kept alive by a `systemd` unit.
- **Reverse proxy**: Nginx forwards `/api/*` to the backend.
- **Public access**: a single domain name or server IP exposes both the website and the API.
- **Transport security**: Let's Encrypt TLS via `certbot -nginx` (recommended; not codified in the repo).

This topology is especially appropriate for the current project because the frontend is a static SPA, the backend is stateless, the API surface is small, there is no authentication subsystem, and the repository already uses relative API paths that fit well behind a reverse proxy.

13.5 Step-by-step manual deployment procedure

13.5.1 1. Server preparation

A deployment server must provide:

- Python 3 and `python3-venv`,
- Node.js (Vite 7 requires at least Node 20.19 or 22.12) and npm,
- Nginx,
- sufficient permissions to install dependencies, build artefacts, and configure services.

Example logical paths used throughout the procedure:

- repository checkout: `/opt/fdc/TFG`
- frontend static site root: `/var/www/fdc`

On Debian / Ubuntu:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip \
                    nodejs npm nginx
```

If the distribution-shipped Node.js is older than the version required by Vite, install a newer release (e.g. via `NodeSource` or `nvm`) before building the frontend.

13.5.2 2. Repository checkout

```
sudo mkdir -p /opt/fdc
sudo chown -R $USER:$USER /opt/fdc
cd /opt/fdc
git clone <repository-url> TFG
```

13.5.3 3. Backend installation, tests and validation

Install the backend in an isolated virtual environment:

```
cd /opt/fdc/TFG/backend
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install -r requirements-dev.txt    # test deps only
```

Run the backend test suite before deploying. Passing tests mean the installed dependencies are usable on this server:

```
pytest
```

Smoke-start the backend manually:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

In a separate terminal, verify the health endpoint:

```
curl http://127.0.0.1:8000/api/v1/health
# Expected response: {"status":"ok"}
```

Stop the manual process once health is green.

13.5.4 4. Backend as a persistent service (systemd)

Create `/etc/systemd/system/fdc-backend.service`:

```
[Unit]
Description=TFG FDC FastAPI backend (uvicorn)
After=network.target

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/opt/fdc/TFG/backend
Environment=PYTHONUNBUFFERED=1
ExecStart=/opt/fdc/TFG/backend/.venv/bin/uvicorn \
          app.main:app --host 127.0.0.1 --port 8000
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activate:

```
sudo systemctl daemon-reload
sudo systemctl enable --now fdc-backend
sudo systemctl status fdc-backend
```

13.5.5 5. Frontend build, tests and publish

Install dependencies, run the test suite, then produce the optimised bundle:

```
cd /opt/fdc/TFG/frontend
npm ci                          # reproducible install from package-lock.json
npm test                        # must exit 0
npm run build                   # emits frontend/dist/
```

Copy the static output to the Nginx document root:

```
sudo mkdir -p /var/www/fdc
sudo rsync -av --delete \
     /opt/fdc/TFG/frontend/dist/ /var/www/fdc/
```

13.5.6 6. Reverse-proxy configuration (Nginx)

Create `/etc/nginx/sites-available/fdc.conf`:

```
server {
    listen 80;
    server_name fdc.example.org;   # replace with your domain

    root /var/www/fdc;
    index index.html;

    # SPA routing: unknown paths fall back to index.html.
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API traffic is proxied to the FastAPI backend.
    location /api/ {
        proxy_pass         http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # Cache immutable Vite bundle assets aggressively.
    location /assets/ {
        expires 30d;
        access_log off;
        add_header Cache-Control "public, max-age=2592000, immutable";
    }
}
```

Enable and reload:

```
sudo ln -s /etc/nginx/sites-available/fdc.conf \
          /etc/nginx/sites-enabled/fdc.conf
sudo nginx -t
sudo systemctl reload nginx
```

13.5.7 7. HTTPS enablement (recommended)

Public deployments should terminate TLS at Nginx. Using `certbot`:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d fdc.example.org
```

Certbot rewrites the Nginx server block to listen on 443 with a trusted certificate and sets up auto-renewal via a systemd timer.

13.6 Repository-specific deployment considerations

13.6.1 CORS configuration

The backend currently whitelists only `http://localhost:5173`. Before publishing, edit `backend/app/main.py` so `allow_origins` matches the production origin exactly, for example:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://fdc.example.org"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

If the frontend and backend are served from the same origin (as in the recommended Nginx reverse-proxy topology), CORS is technically unnecessary, but keeping the whitelist aligned with the canonical domain is still good hygiene.

13.6.2 Frontend API path behaviour

The frontend uses relative paths (`/api/v1/compute`). This is beneficial for deployment: the compiled bundle does not need to know the backend URL, and Nginx handles routing transparently.

13.6.3 Development proxy vs. production routing

The Vite proxy only applies during `npm run dev`. In production, Nginx is the sole traffic router; a deployment that serves only the static frontend files without configuring `/api/` routing will produce browser 404s.

13.6.4 Encoding of `requirements.txt`

In some snapshots the file may have been saved as UTF-16 (note the BOM and wide spacing). If `pip install -r requirements.txt` misbehaves, rewrite the file as UTF-8 (one package per line, no BOM) before deployment.

13.6.5 Local development artefacts

`backend/.venv/`, `frontend/node_modules/` and `frontend/dist/` should *not* be treated as authoritative deployment artefacts. A clean deployment rebuilds the virtualenv and the bundle from the declared manifest files:

- `backend/requirements.txt`
- `backend/requirements-dev.txt`
- `frontend/package.json` + `frontend/package-lock.json`

13.7 Release procedure

Recommended manual release checklist:

1. Fetch the latest commits on the server (`git -C /opt/fdc/TFG pull`).
2. Recreate or update the backend virtualenv if `requirements.txt` changed.
3. `pip install -r requirements.txt` (and `requirements-dev.txt` if you want to run the tests on the server).
4. Run the backend test suite: `cd backend && pytest`.
5. `npm ci` in `frontend/` if `package-lock.json` changed.
6. Run the frontend test suite: `npm test`.
7. Generate a fresh bundle: `npm run build`.
8. Publish the static files: `sudo rsync -av -delete frontend/dist/ /var/www/fdc/`.
9. Restart the backend service: `sudo systemctl restart fdc-backend`.
10. Reload Nginx if its config changed: `sudo systemctl reload nginx`.
11. Run the post-release smoke test (see below).

Post-release smoke test

1. `curl https://fdc.example.org/api/v1/health` returns `{"status":"ok"}`.
2. Load the site in a browser: the page renders, the input panel prefills on the `40 cm` preset, and "Run Statistical Fit" succeeds.
3. The classification card, metrics table and chart populate with the expected four curves and scatter points.
4. "Export High-Res PNG" downloads a non-empty PNG.
5. "Export PDF Report" downloads a non-empty PDF.

13.8 Rollback considerations

Because the application is stateless (no database, no schema migrations), rollback is simple:

- Keep the previously deployed `dist/` directory under a timestamped backup (e.g. `/var/www/fdc.backup-202` before replacing it.
- Tag every production release in git (e.g. `git tag prod-2026-04-19 && git push -tags`).
- If a release fails, `rsync` the backup back into `/var/www/fdc/` and check out the previous tag on the server before restarting the backend.

13.9 Summary

The repository supports deployment as a conventional two-tier web application composed of:

- a static Vite frontend served by Nginx,
- a FastAPI backend served by Uvicorn under systemd,
- and a reverse proxy joining both under one public origin, with TLS terminated at Nginx.

The deployment process is fully manual today. The absence of CI/CD, containers and infrastructure-as-code means that reproducibility depends on careful operator execution. Because the application is stateless, compact and based on standard technologies, it remains straightforward to deploy on a standard Linux server with Nginx and Uvicorn; the test suites (§11) provide a fast pre-flight check to catch environment regressions before publishing.

14 Troubleshooting

- **404 on `/api/v1/compute` from the SPA.** In development the Vite proxy is hard-coded to `http://localhost:8000`; ensure the backend is running there. In production, ensure Nginx's `location /api/` block is enabled.
- **CORS error in the browser.** The backend whitelists only `http://localhost:5173` by default. Update `allow_origins` in `backend/app/main.py` if you use a different host, port, or production domain.
- **GEKKO install or runtime errors.** GEKKO ships a platform-specific solver; ensure your Python / OS match a supported wheel.
- **`400 Expected exactly 7 y values / All y values must be finite numbers.`** Fill all seven fields with finite numeric values; empty, non-numeric, NaN or Infinity entries are rejected.
- **Stale computation shown after changing inputs.** The SPA uses a version counter to discard stale responses (see `computationVersionRef` in `App.tsx`); rerun the fit if one slips through.
- **Ports 5173 / 8000 already in use.** Stop the conflicting process or pass `-port` to `vite` / `uvicorn`, and update the corresponding hardcoded origin / proxy.
- **Tests fail on first run.** Make sure the correct interpreter is active (`which python`, `which node`) and that `pip install -r requirements-dev.txt` / `npm ci` were run from their respective directories.

15 Conclusion

The codebase delivers a coherent, self-contained implementation of a Fixation Disparity Curve analysis tool: a small but typed and validated FastAPI backend that performs constrained nonlinear fits with GEKKO, and a focused React 19 + Vite SPA that renders the results, surfaces the clinical summary, and supports PNG / PDF export. The codebase is intentionally lean — no database, no authentication, no containerisation, no CI — which keeps the current scope aligned with the requirements captured during the initial phase. Automated tests on both sides (26 backend + 56 frontend) safeguard the numerical core, the HTTP contract, and the UI helpers against regressions during deployment and future evolution.